

Lecture 9

Recurrent Neural Networks

Sequential vs. Non-Sequential Data

- Non-sequential data (非序列化数据) does not have an inherent order and can be processed independently of other data points
 - tabular data
 - images

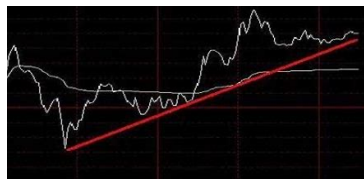
编号	色泽	根蒂	敲声
1	青绿	蜷缩	浊响
2	乌黑	蜷缩	沉闷
3	乌黑	蜷缩	浊响
4	青绿	蜷缩	沉闷
5	浅白	蜷缩	浊响
6	青绿	稍蜷	浊响
7	乌黑	稍蜷	浊响
8	乌黑	稍蜷	浊响



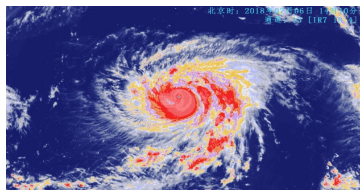
Sequential vs. Non-Sequential Data

Sequential data (序列化数据) refers to data that has an inherent order and depends on previous elements in the sequence.

- Time series data
- Speech signals
- Text



Stock price



Meteorological data
气象数据



Video



Electrocardiogram 心电图



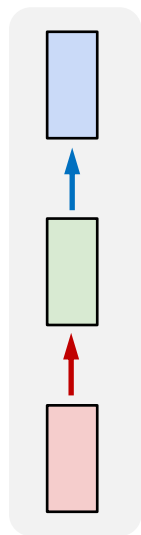
Seismic data
地震数据



Speech signals

“Vanilla” Neural Network

one to one

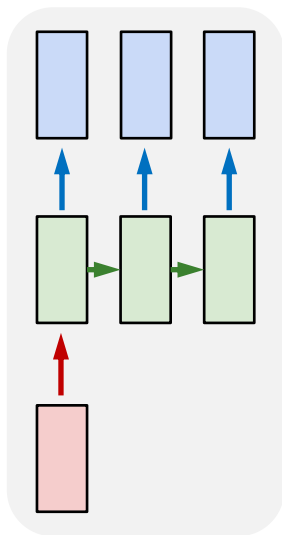
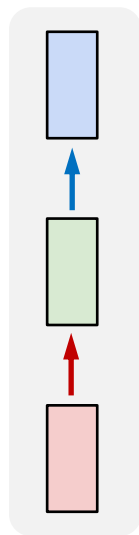


Vanilla Neural Networks

Recurrent Neural Networks: Process Sequences

one to one

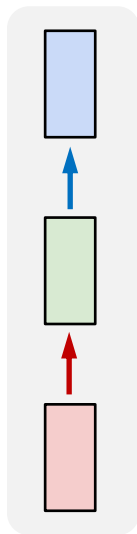
one to many



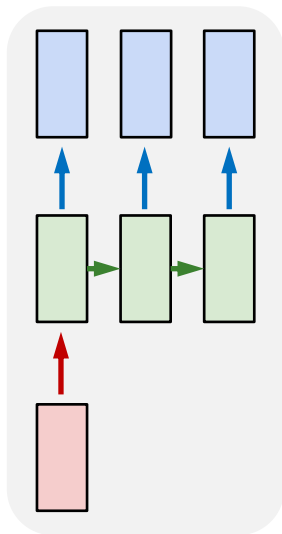
e.g. Image Captioning
image -> sequence of words

Recurrent Neural Networks: Process Sequences

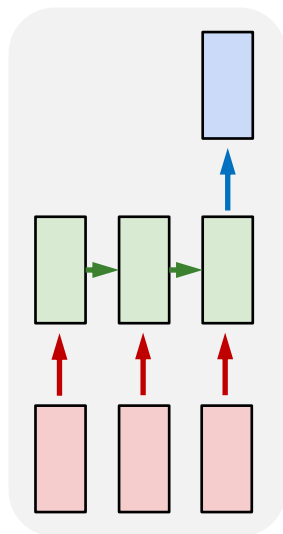
one to one



one to many



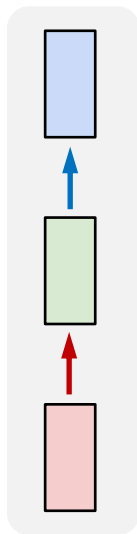
many to one



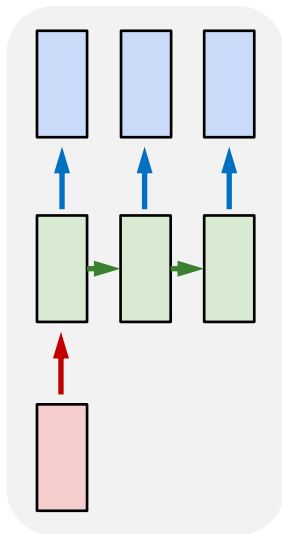
e.g. action prediction
sequence of video frames -> action class

Recurrent Neural Networks: Process Sequences

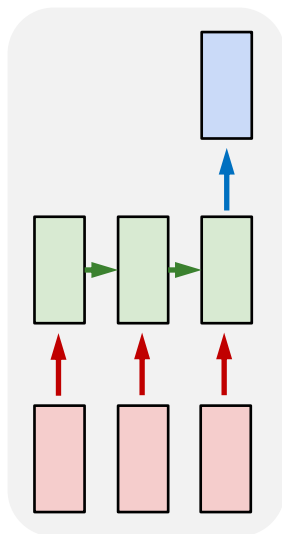
one to one



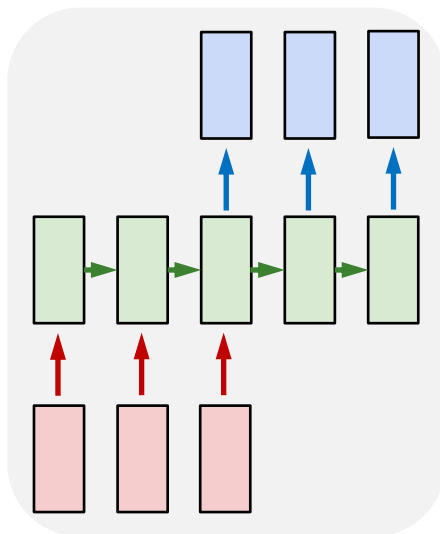
one to many



many to one



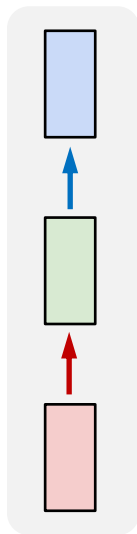
many to many



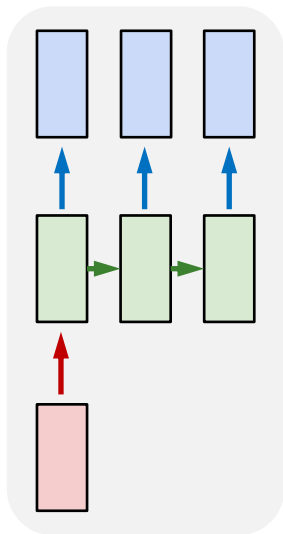
e.g. Video Captioning
Sequence of video frames -> caption

Recurrent Neural Networks: Process Sequences

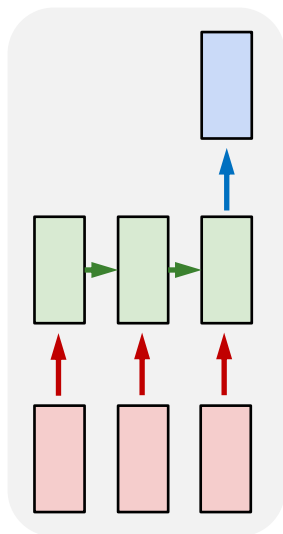
one to one



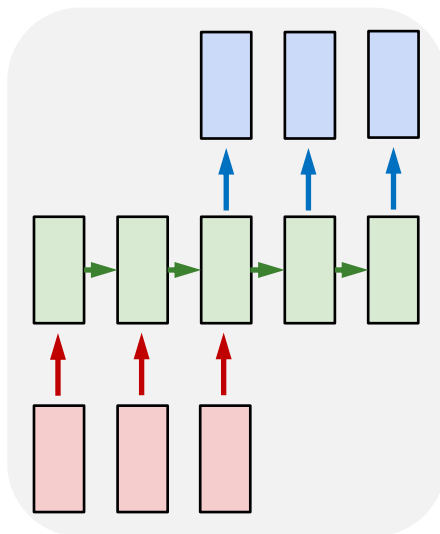
one to many



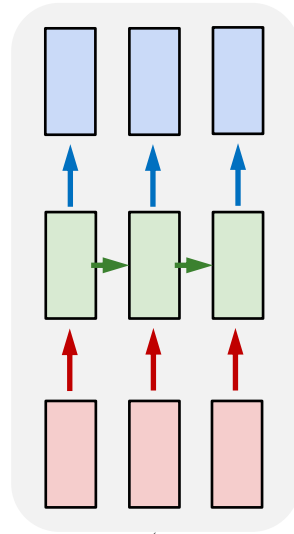
many to one



many to many

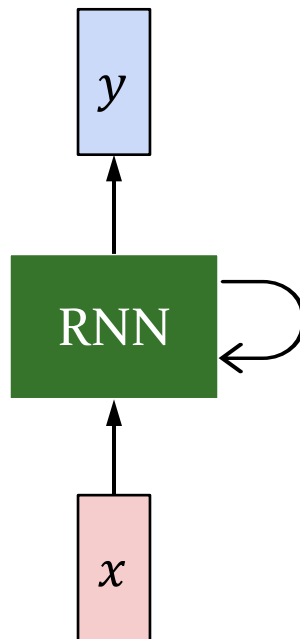


many to many

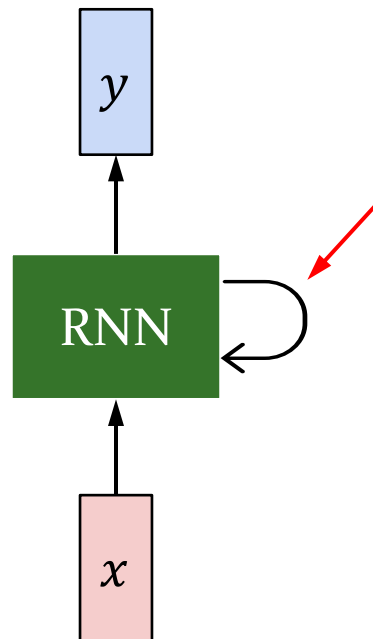


e.g. Video classification on frame level

Recurrent Neural Network

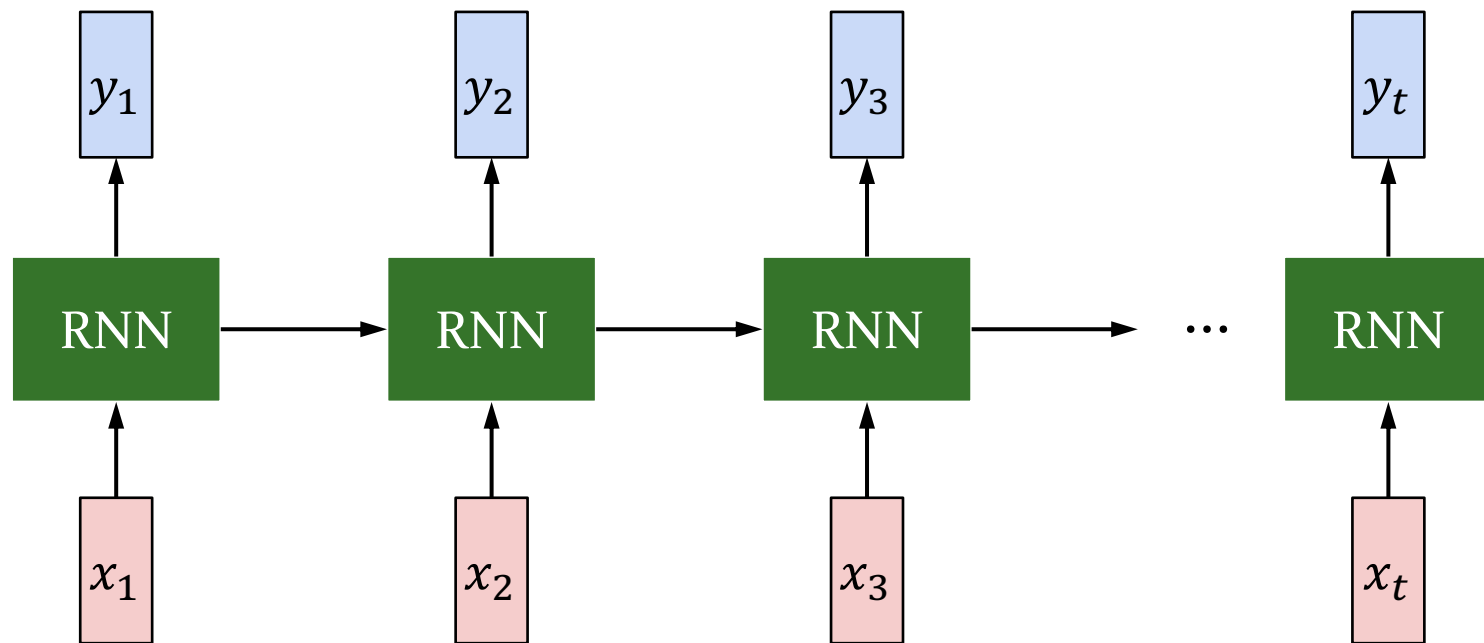


Recurrent Neural Network



Key idea: RNNs have an “internal state” that is updated as a sequence is processed

Unrolled RNN



RNN hidden state update

- We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

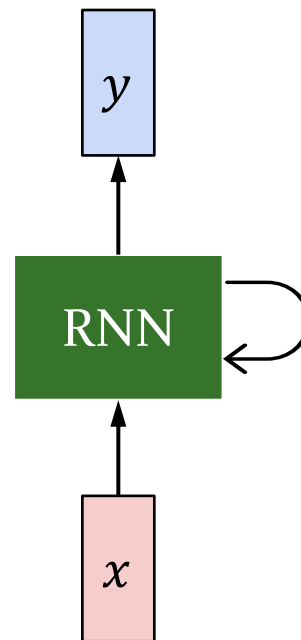
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

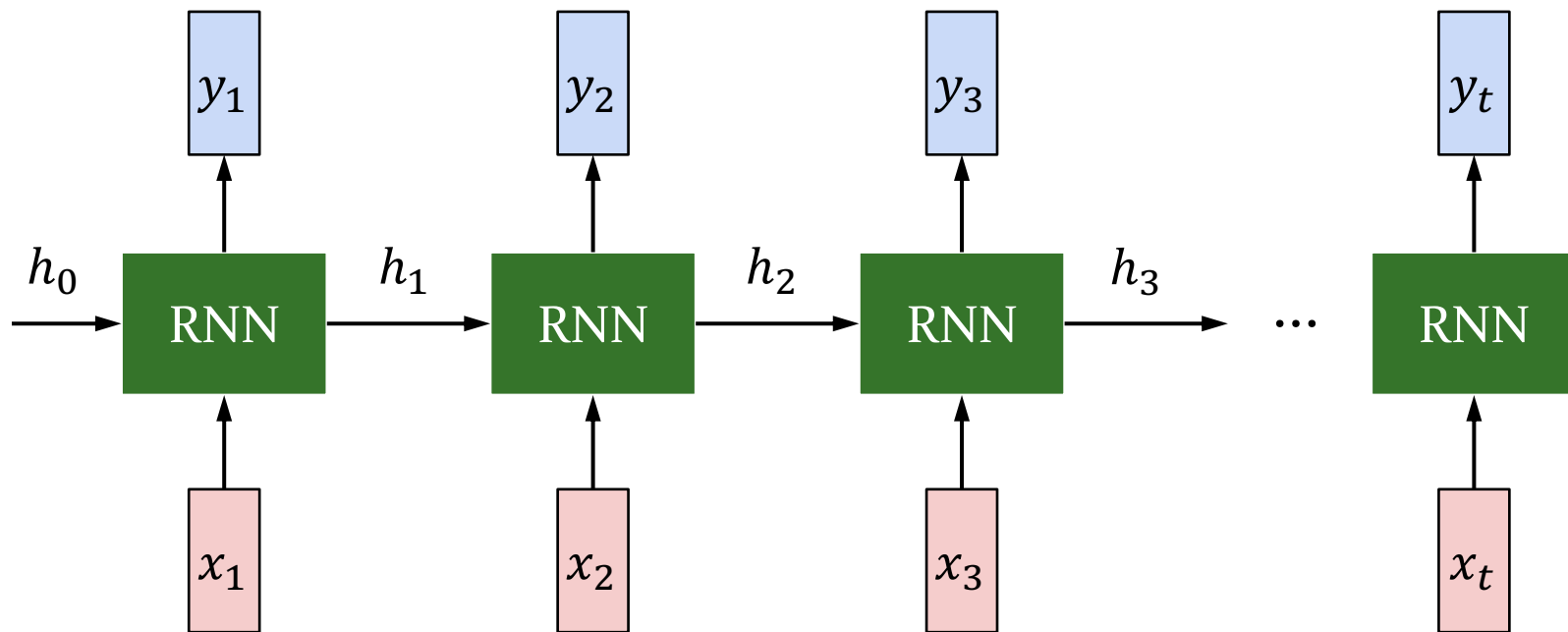
some function with parameter W

old state

input vector at time step t



Recurrent Neural Network

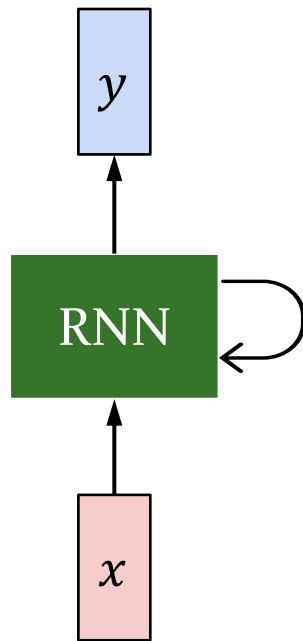


Recurrent Neural Network

- We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

Notice: the same function and the same set of parameters are used at every time step.
(权重共享, 时间不变性)

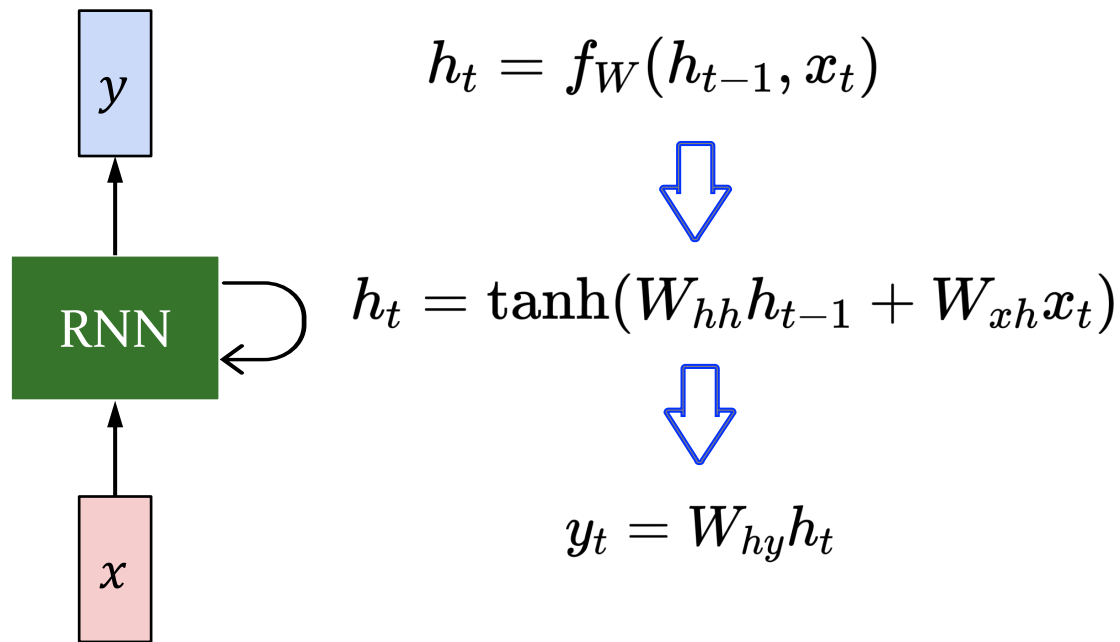


CNN 和 RNN 中的权重共享对比

	CNN (卷积神经网络)	RNN (循环神经网络)
假设	平移不变性	时间不变性
的数据结构	网格状数据（如图像、网格数据）	序列数据（如文本、时间序列、语音）
共享的含义	同一个特征检测器（卷积核）在输入的不同位置滑动，检测相同的模式。	同一套状态转移规则（权重矩阵）在序列的每个时间步上重复应用。
比喻	用同一把尺子测量图片的不同区域。无论猫出现在图片的左上角还是右下角，都用同一个“猫过滤器”去识别。	用同一套物理定律预测物体的下一个状态。无论是第1秒还是第100秒，物体运动都遵循牛顿定律（相同的状态转移方程）。
的权重	卷积核的参数	循环单元的参数（如 W_{hh} , W_{xh} ）
优势	1. 参数效率：无需为每个像素位置学习独立参数。 2. 平移等价性：模式无论出现在哪里都能被识别。	1. 参数效率：可处理任意长序列。 2. 捕捉时序动态：将过去信息编码到隐藏状态中。

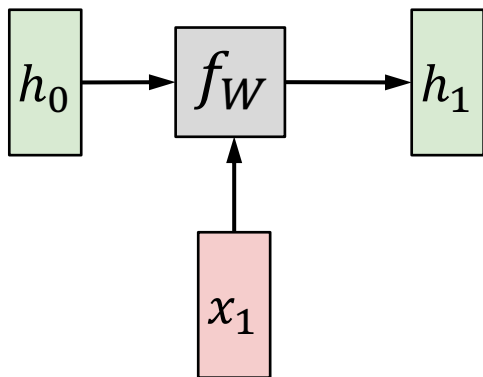
(Vanilla) Recurrent Neural Network

The state consists of a single “hidden” vector $\mathbf{h}$:

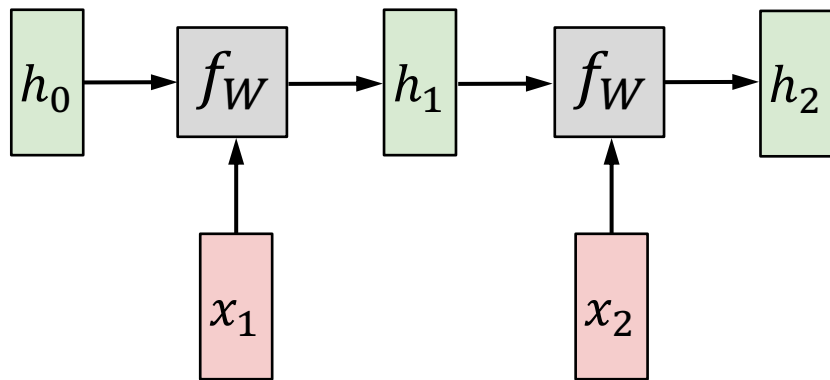


$$\tanh x = \frac{e^{2x} - 1}{e^{2x} + 1}$$
$$\frac{d}{dx} \tanh x = 1 - \tanh^2 x$$

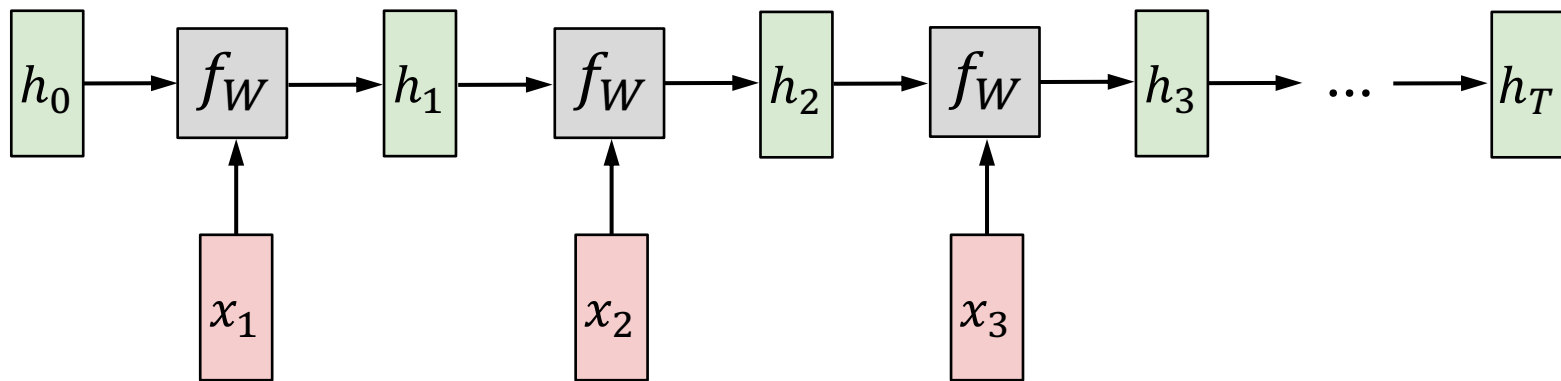
RNN: Computational Graph



RNN: Computational Graph

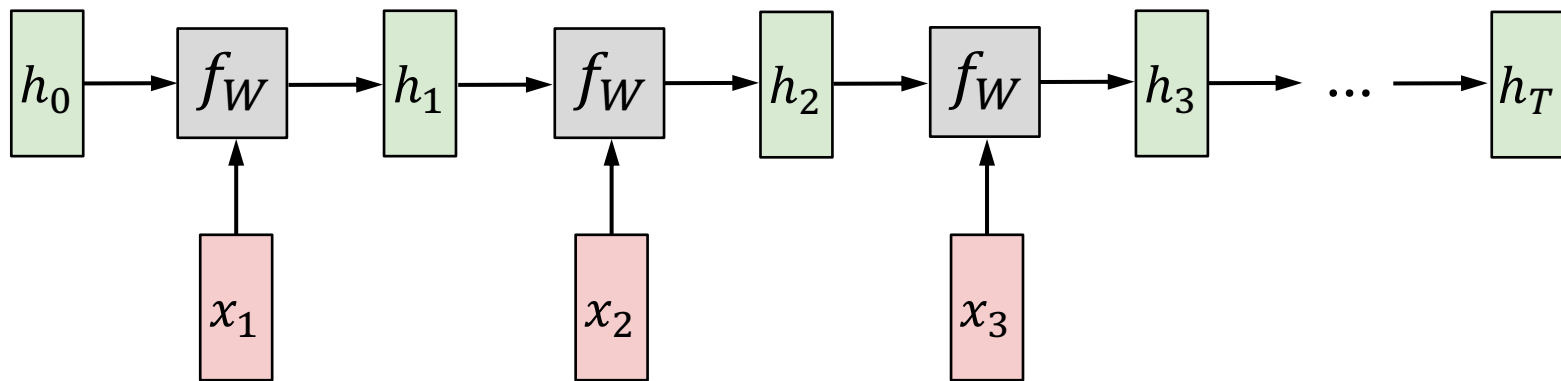


RNN: Computational Graph

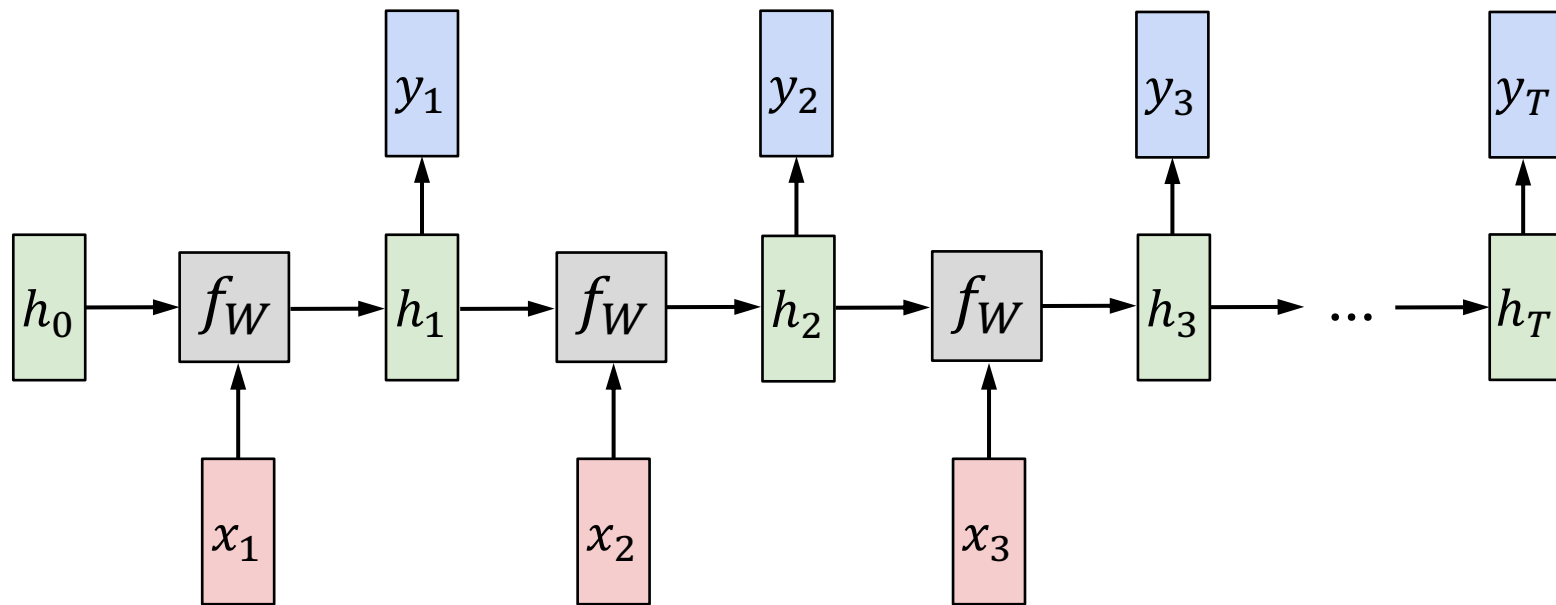


RNN: Computational Graph

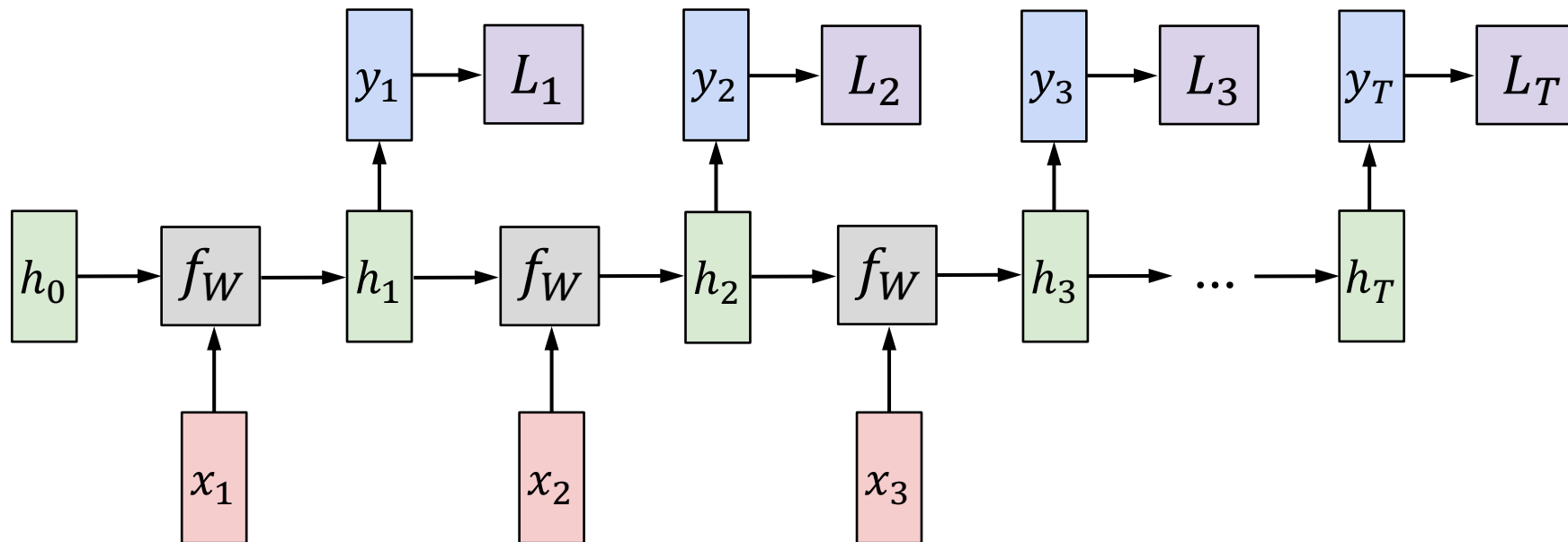
Re-use the same weight matrix at every time-step



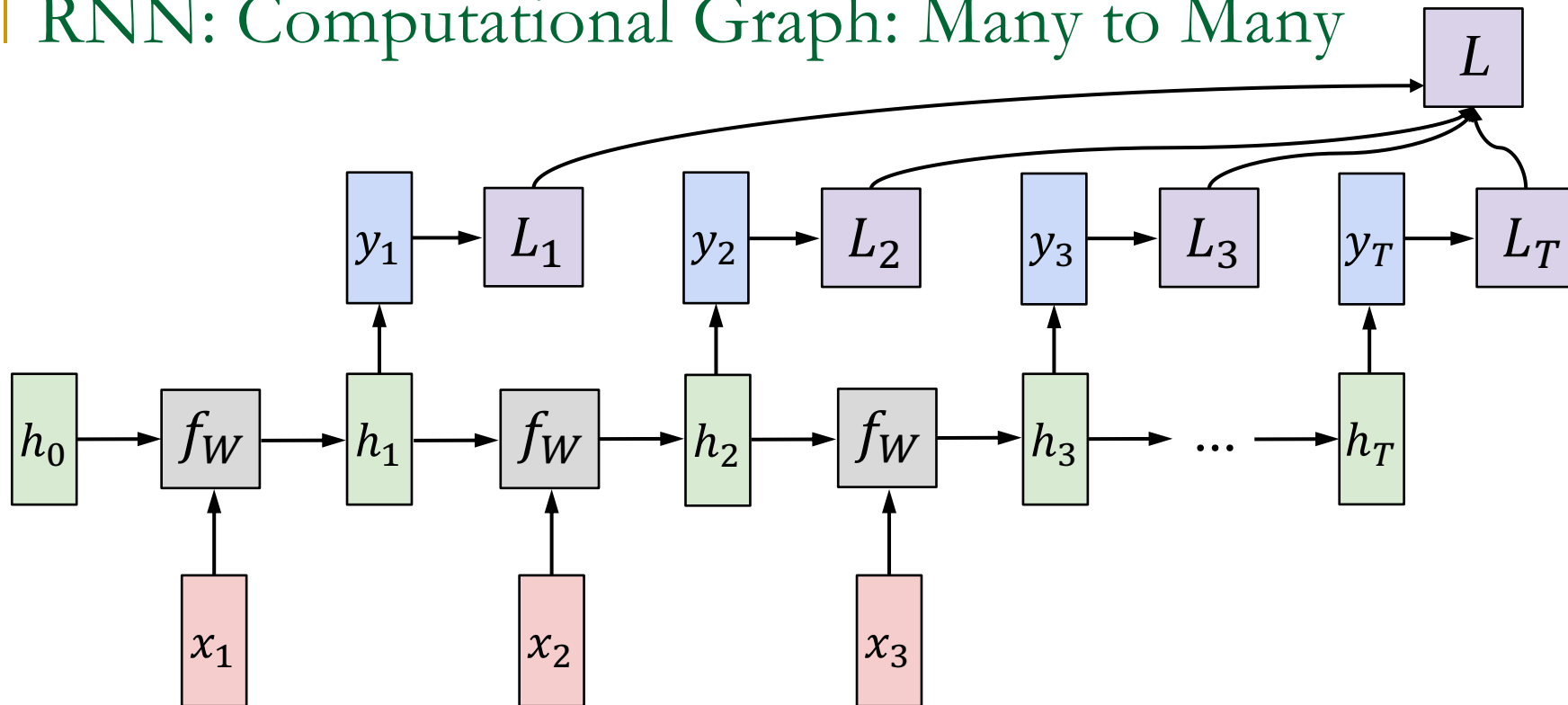
RNN: Computational Graph: Many to Many



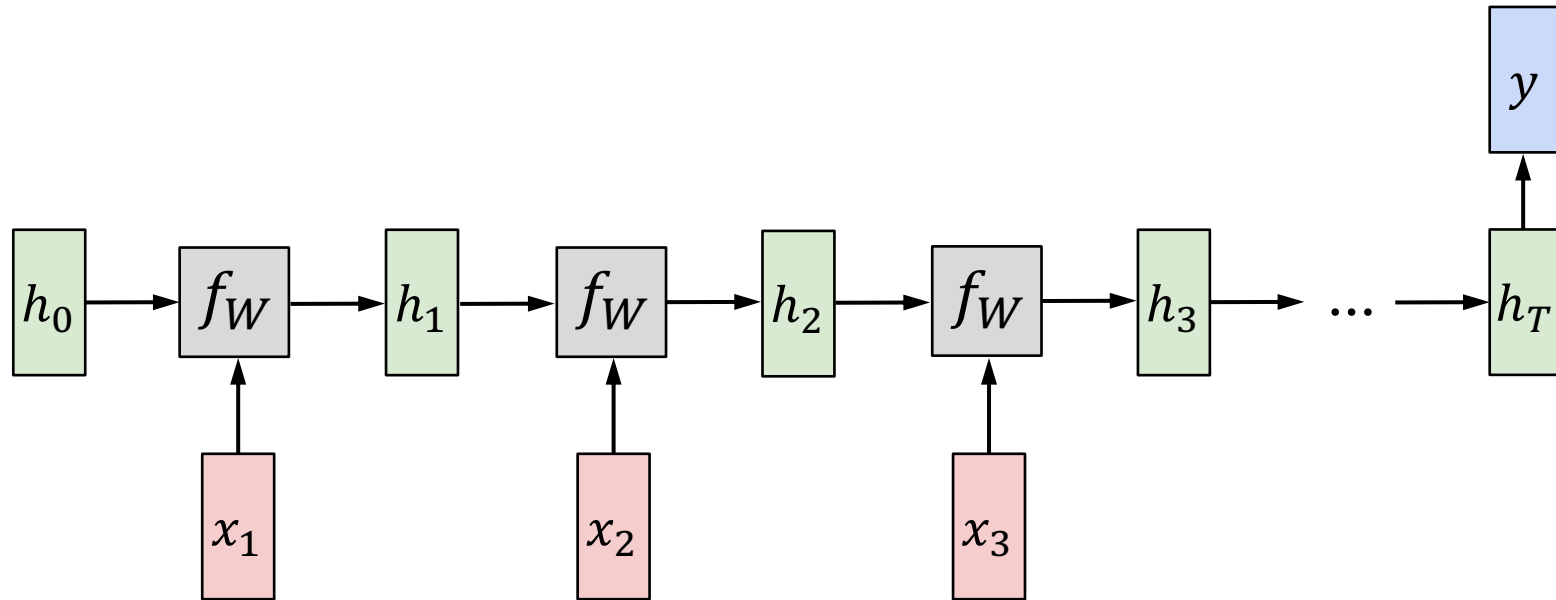
RNN: Computational Graph: Many to Many



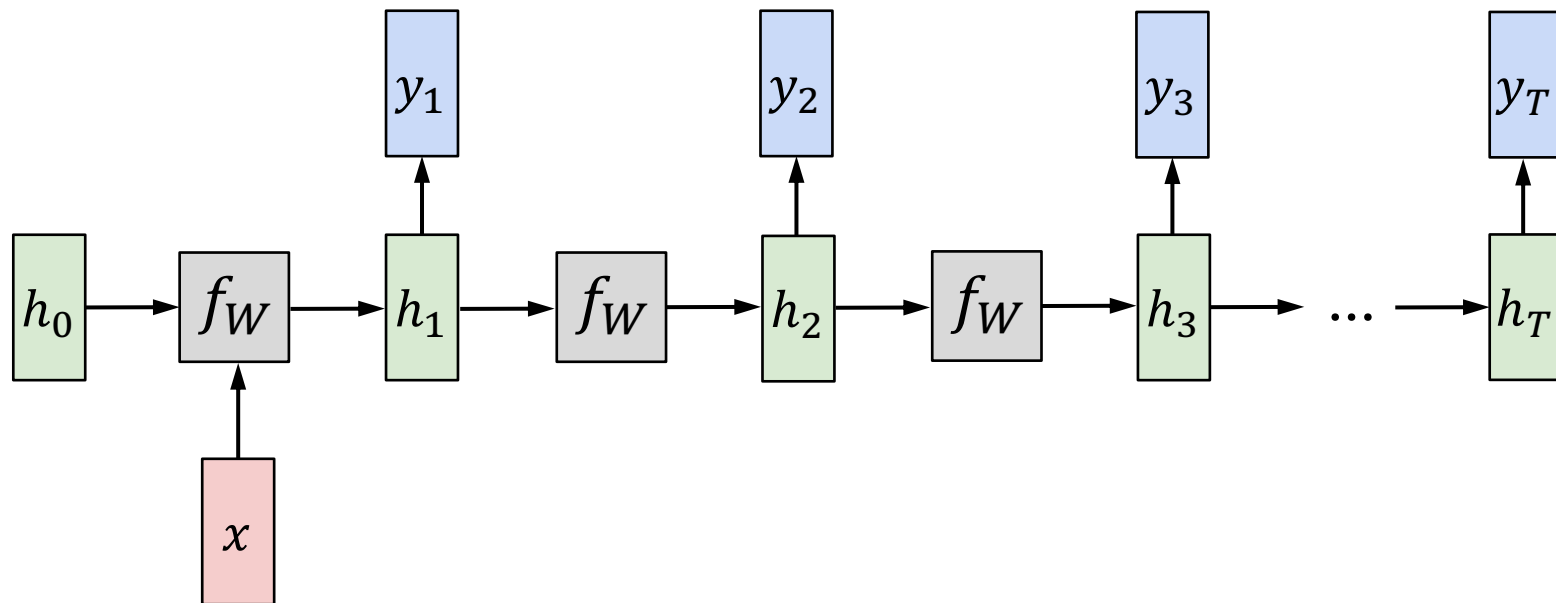
RNN: Computational Graph: Many to Many



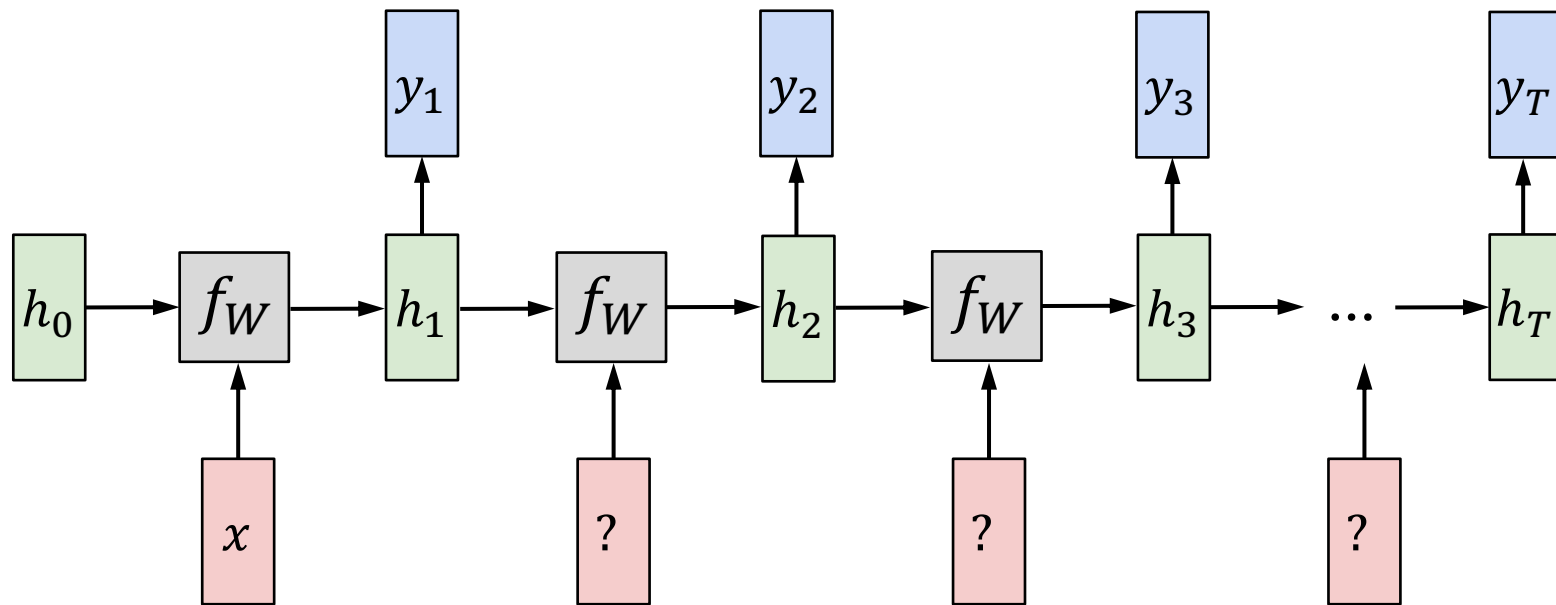
RNN: Computational Graph: Many to One



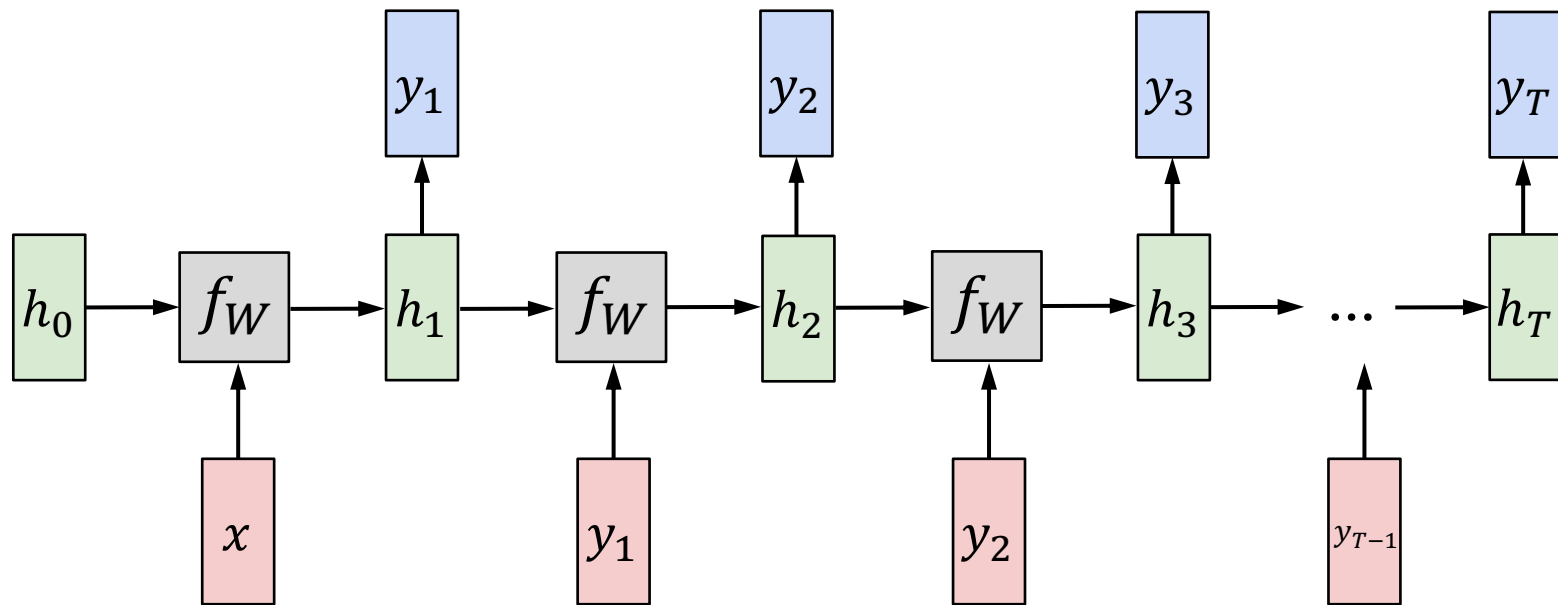
RNN: Computational Graph: One to Many



RNN: Computational Graph: One to Many

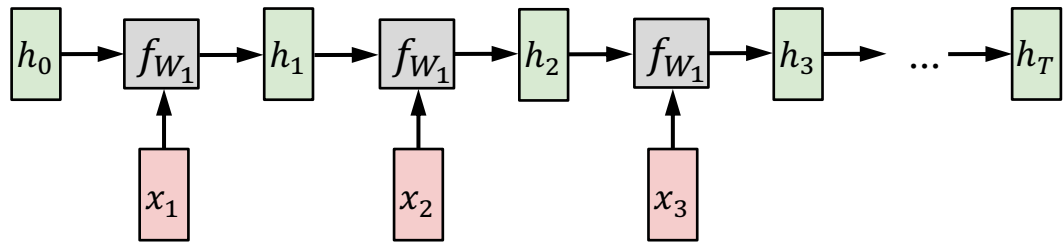


RNN: Computational Graph: One to Many

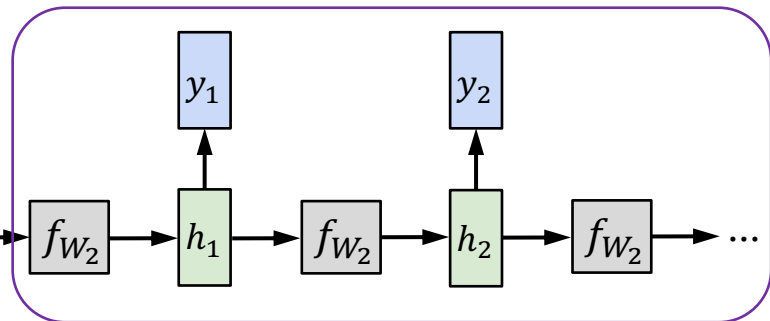


Sequence to Sequence: Many-to-one + one-to-many

Many to one: Encode input sequence in a single vector



One to many: Produce output sequence from single input vector

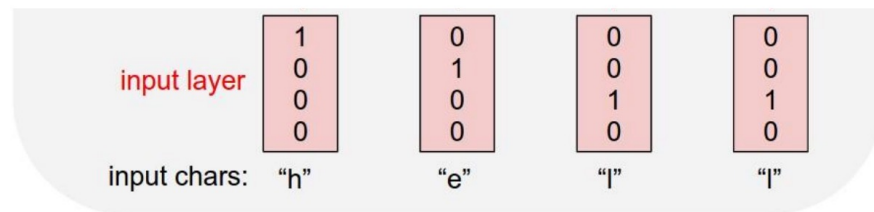


Sutskever et al, "Sequence to Sequence Learning with Neural Networks", NIPS 2014

Example: Character-level Language Model

Vocabulary: [h,e,l,o]

Example training sequence: “**hello**”

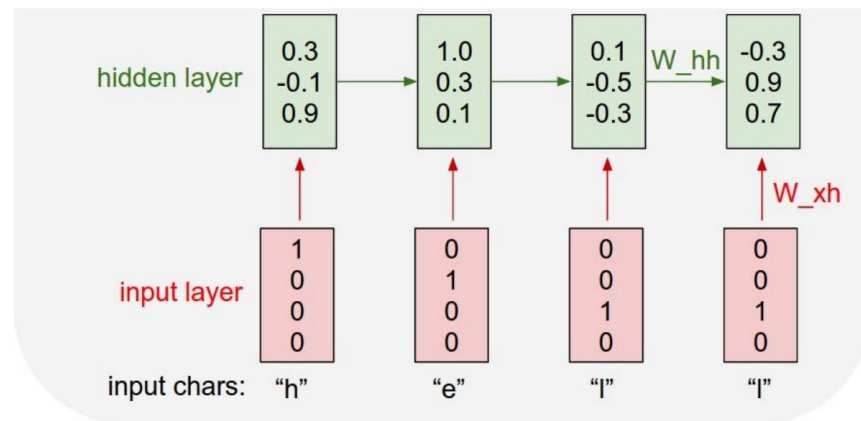


Example: Character-level Language Model

Vocabulary: [h,e,l,o]

Example training sequence: “**hello**”

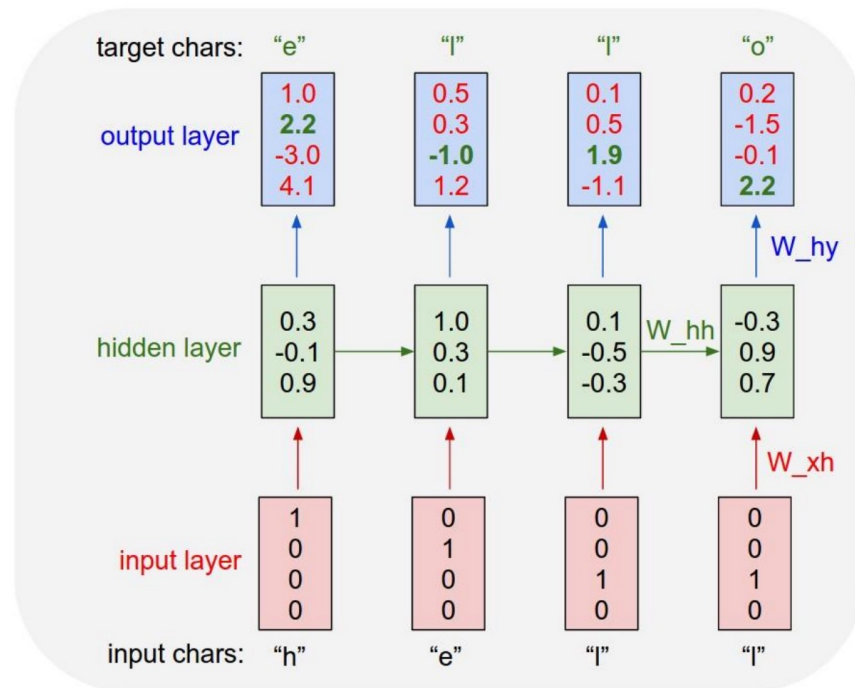
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



Example: Character-level Language Model

Vocabulary: [h,e,l,o]

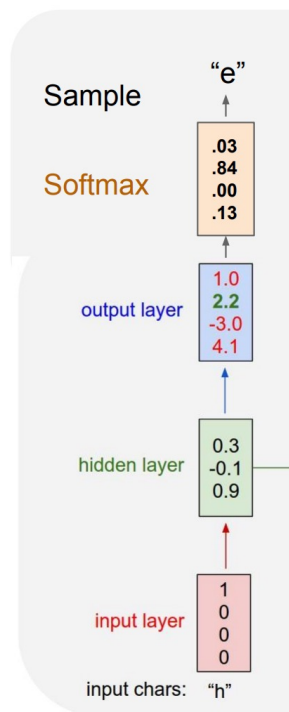
Example training sequence: “hello”



Example: Character-level Language Model

Vocabulary: [h,e,l,o]

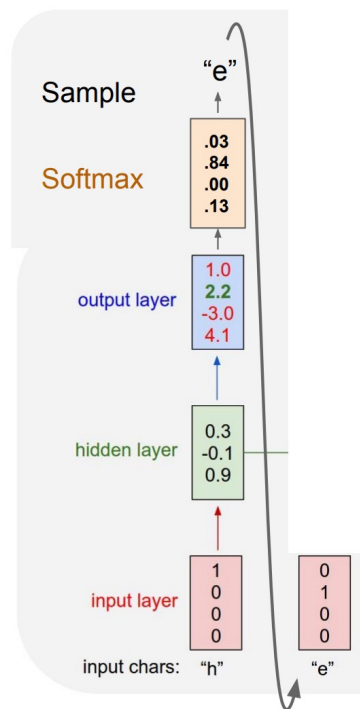
At test-time sample characters one at a time, feed back to model



Example: Character-level Language Model

Vocabulary: [h,e,l,o]

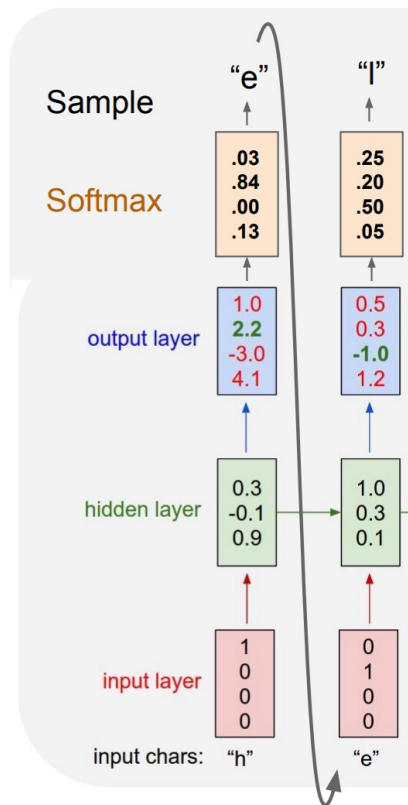
At test-time sample characters one at a time, feed back to model



Example: Character-level Language Model

Vocabulary: [h,e,l,o]

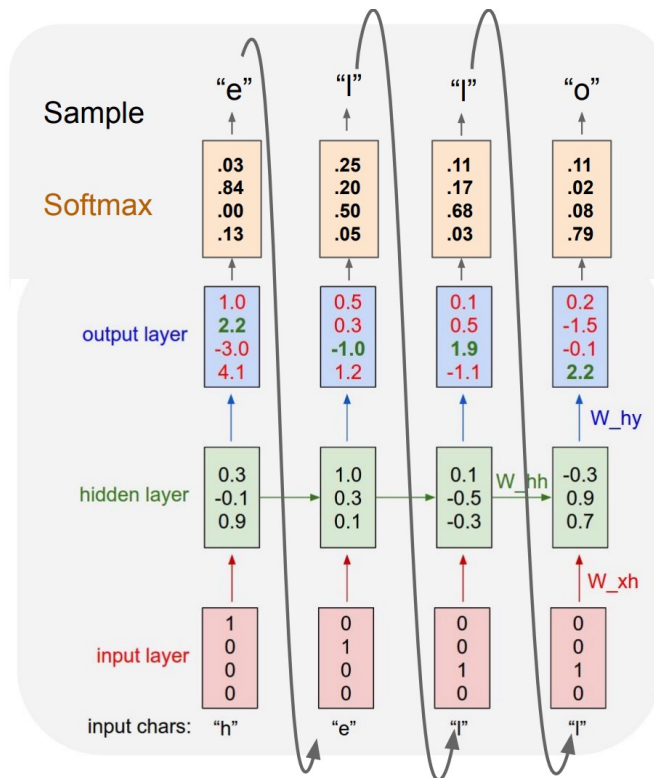
At test-time sample characters one at a time, feed back to model



Example: Character-level Language Model

Vocabulary: [h,e,l,o]

At test-time sample characters one at a time, feed back to model

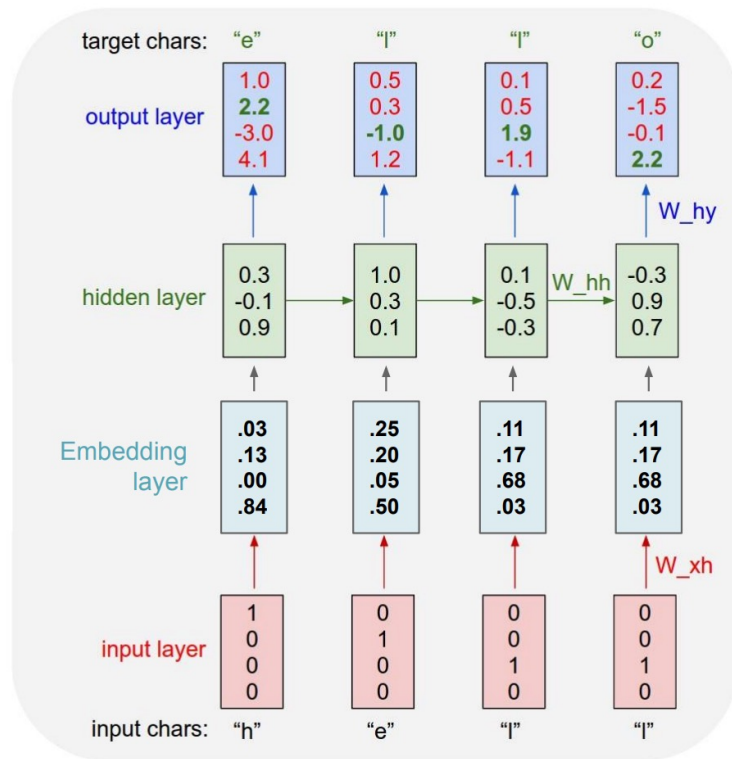


Example: Character-level Language Model

Vocabulary: [h,e,l,o]

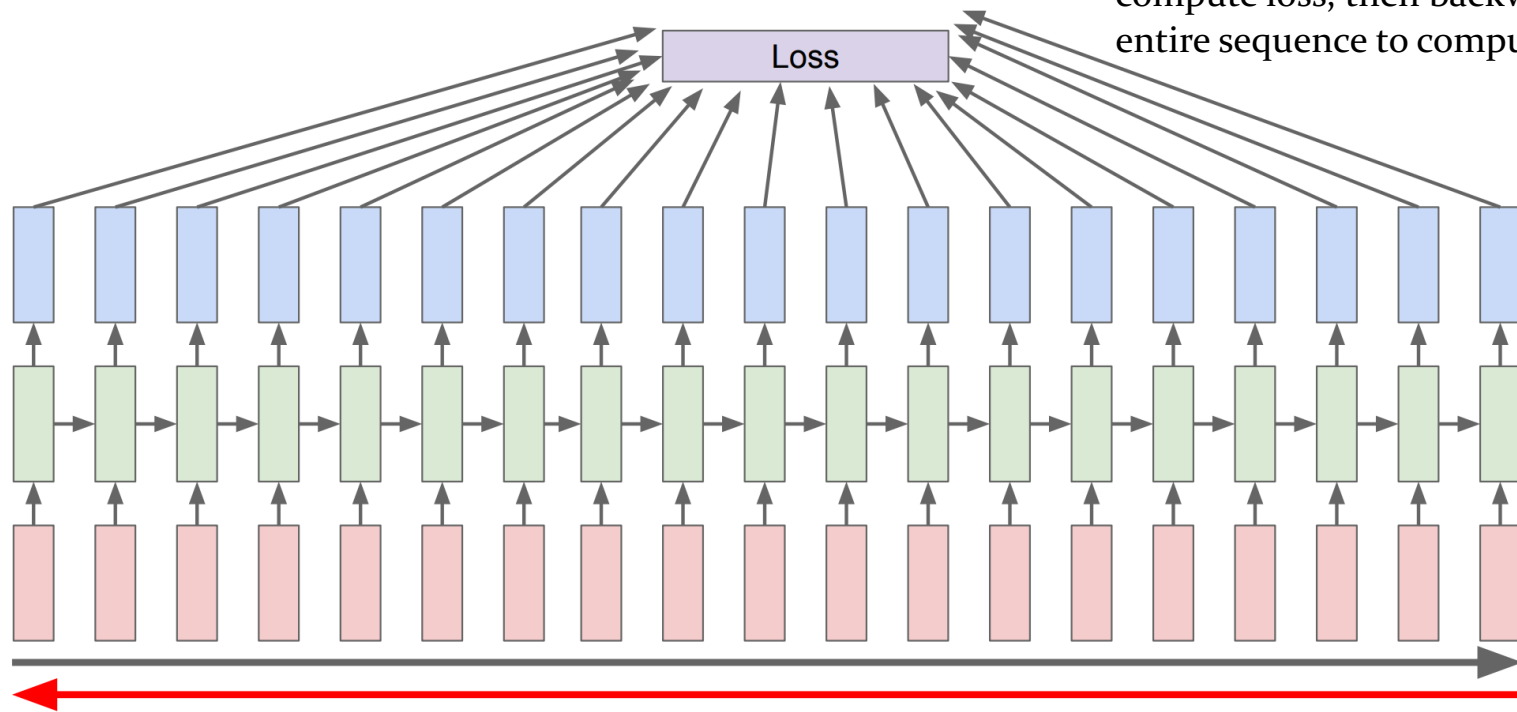
Matrix multiply with a one-hot vector just extracts a column from the weight matrix. We often put a separate embedding layer between input and hidden layers.

$$\begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} w_{11} \\ w_{21} \\ w_{31} \end{bmatrix}$$



Backpropagation through time (BPTT)

Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient



Backpropagation through time (BPTT)

- ◆ BPTT is an adaptation of the standard backpropagation algorithm for RNNs, taking into account the temporal dependencies and recurrent connections in the network.
- ◆ The goal is to minimize the loss function by adjusting the network's weights through gradient descent.
- ◆ BPTT involves computing the gradients of the loss function with respect to the weights, considering the contributions of all time steps in the sequence.

RNN tradeoffs

■ RNN Advantages:

- ❑ Can process any length input
- ❑ Computation for step t can (in theory) use information from many steps back
- ❑ Model size doesn't increase for longer input
- ❑ Same weights applied on every timestep, so there is symmetry in how inputs are processed.

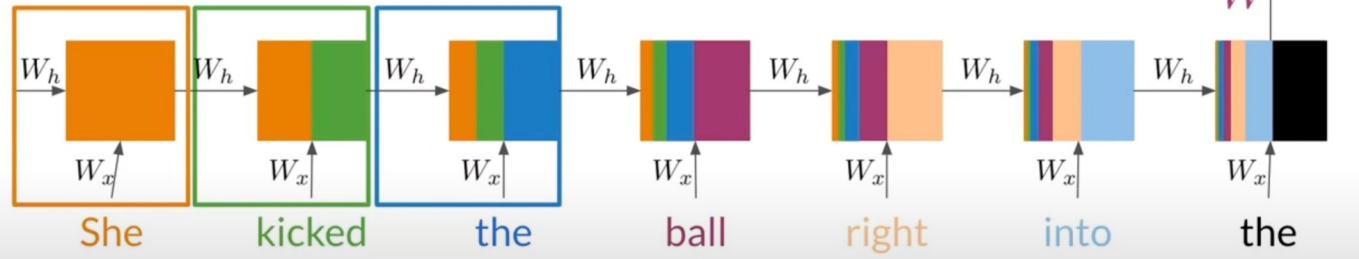
■ RNN Disadvantages:

- ❑ Recurrent computation is slow
- ❑ In practice, difficult to access information from many steps back

Vanilla RNN Vanishing Gradients

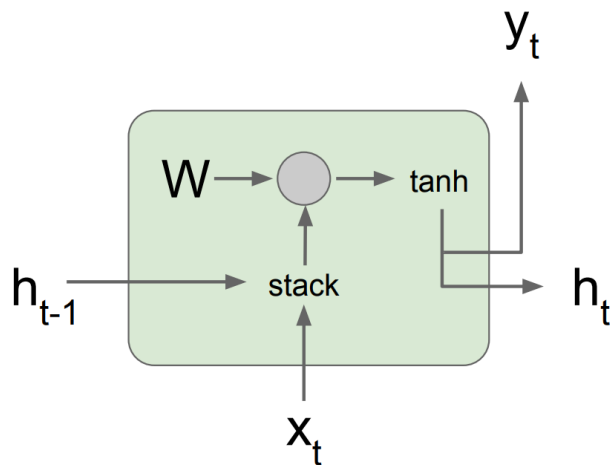
RNN Basic Structure

She kicked the ball right into the _____



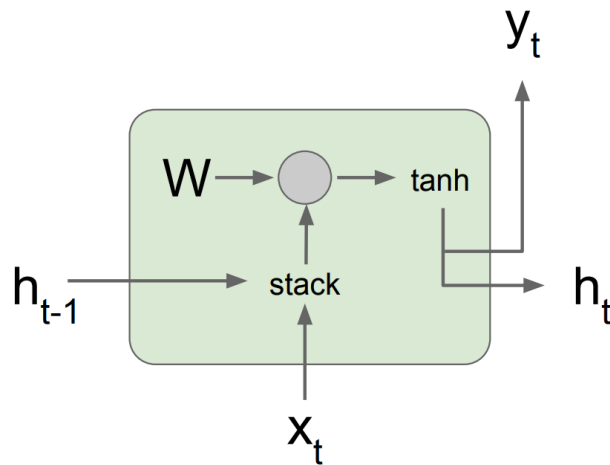
- $t=2$ 时刻, t_1 时刻的信息占50%,
- $t=3$ 时刻, t_2 时刻的总信息占50%, 即 t_1 和 t_2 时刻单独的信息各占25%
- $t=4$ 时刻, t_3 时刻的总信息占50%, 即 t_3 时刻单独的信息占25%, t_1 和 t_2 时刻的信息各占12.5%
- $t=5$ 时刻, t_4 时刻的总信息占50%, 即 t_4 时刻单独的信息占25%, t_3 时刻单独的信息占12.5%, t_1 和 t_2 时刻单独的信息各占6.25%

Vanilla RNN Gradient Flow



$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

Vanilla RNN Gradient Flow

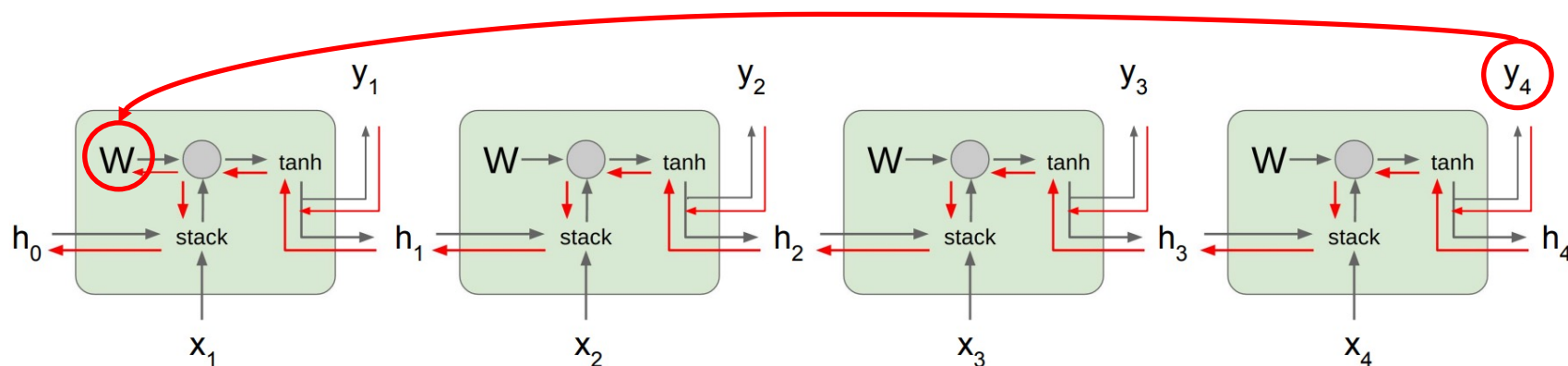


$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{hx}x_t)W_{hh}$$

Vanilla RNN Gradient Flow

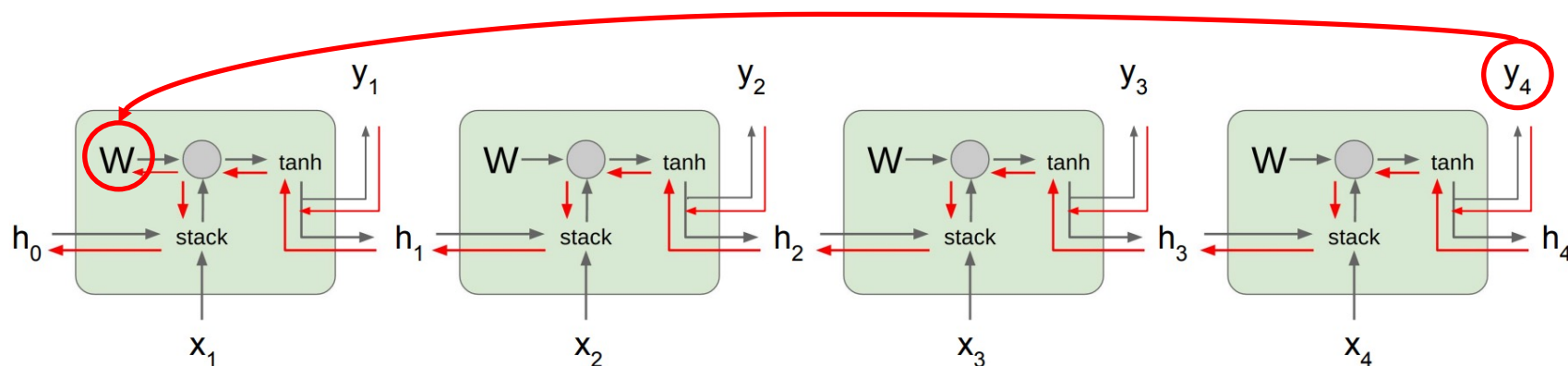
Gradients over multiple time steps:



$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

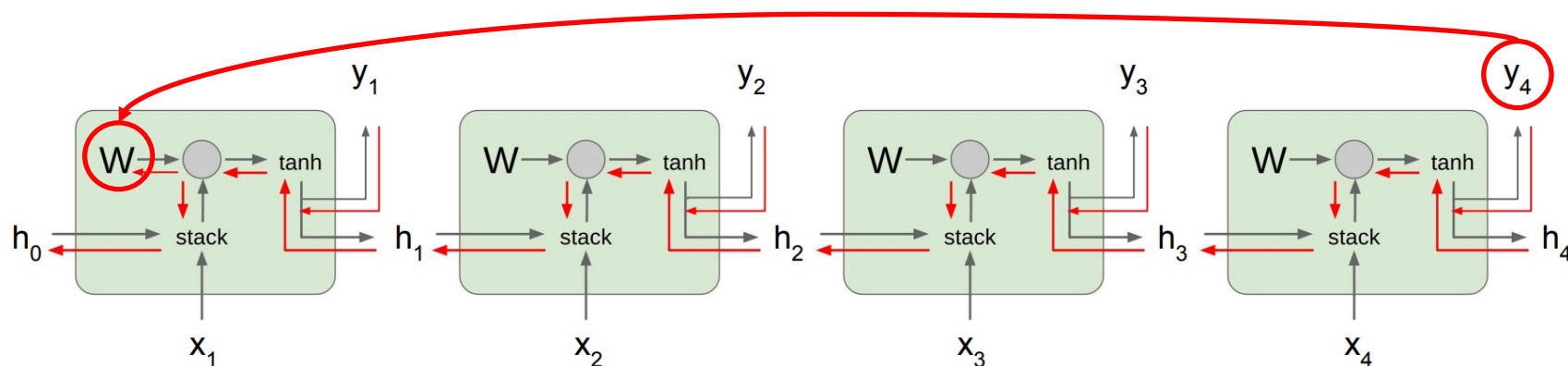
Gradients over multiple time steps:



$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

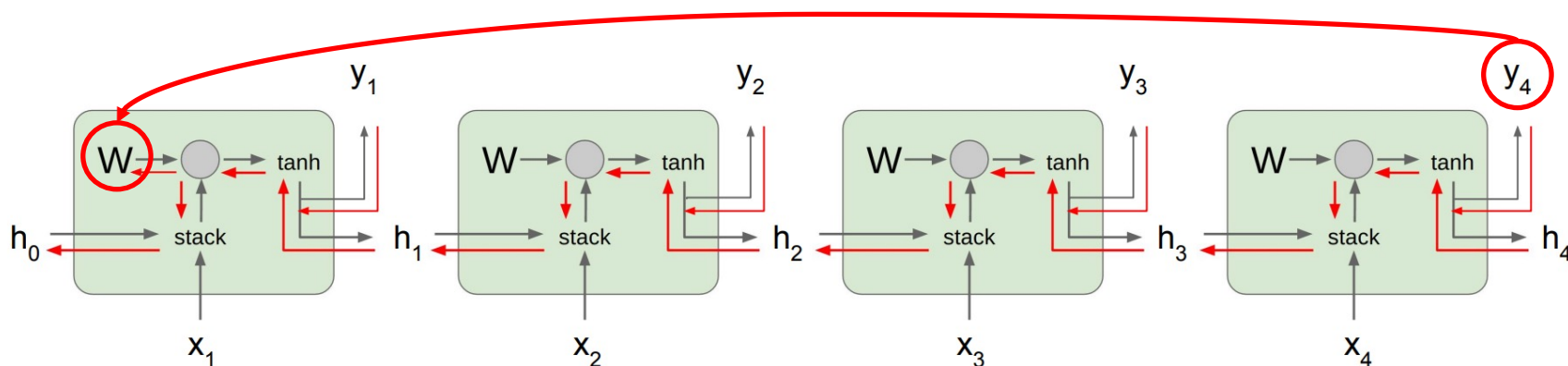


$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{xh}x_t)W_{hh}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

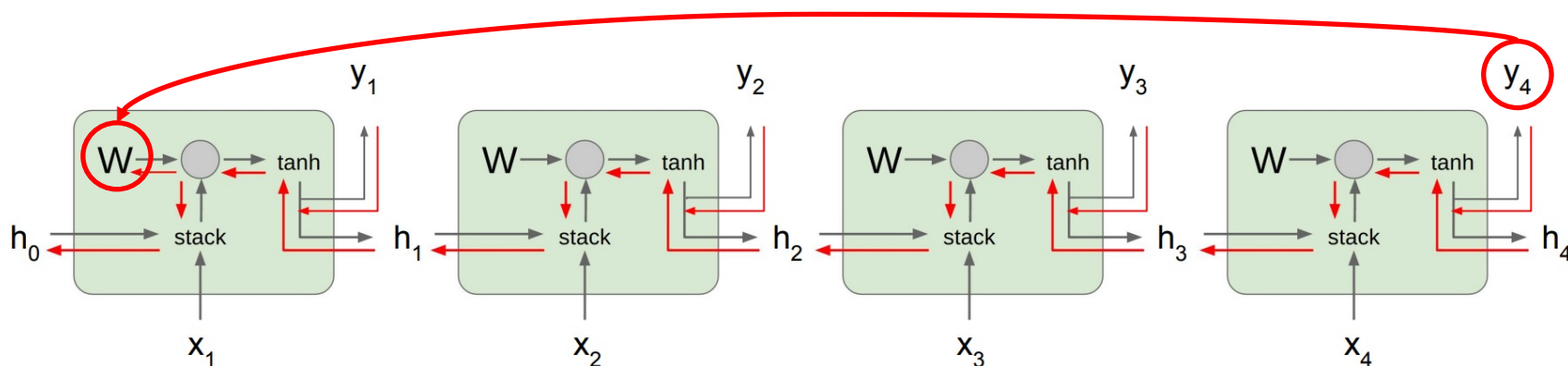


Almost always < 1
Vanishing gradients

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \boxed{\tanh'(W_{hh}h_{t-1} + W_{hx}x_t)} \right) W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:



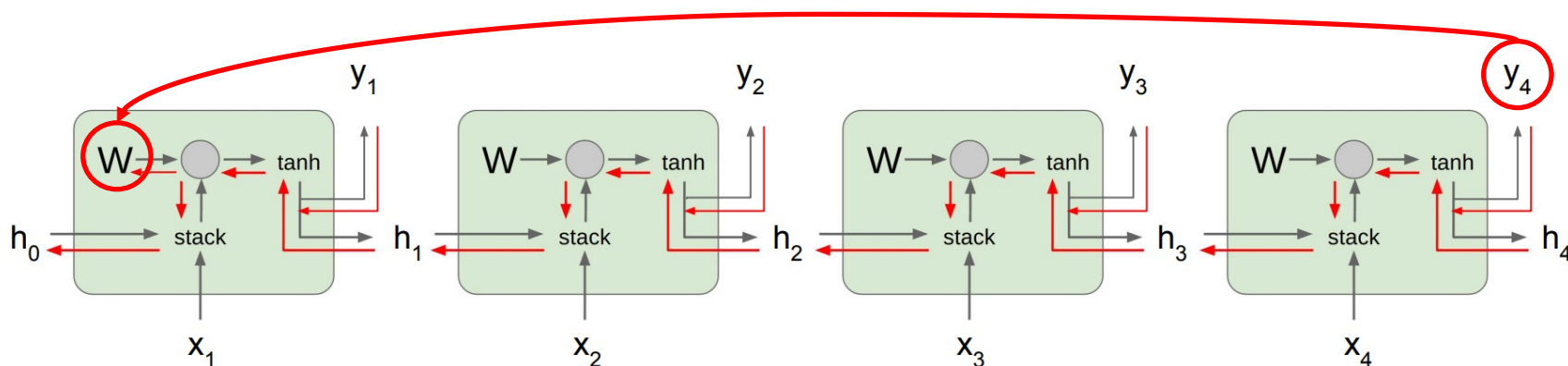
Largest singular value > 1 : **Exploding gradients**

Largest singular value < 1 : **Vanishing gradients**

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \tanh'(W_{hh} h_{t-1} + W_{hx} x_t) \right) \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:



Largest singular value > 1: Exploding gradients → **Gradient clipping:**
Scale gradient if its norm is too big

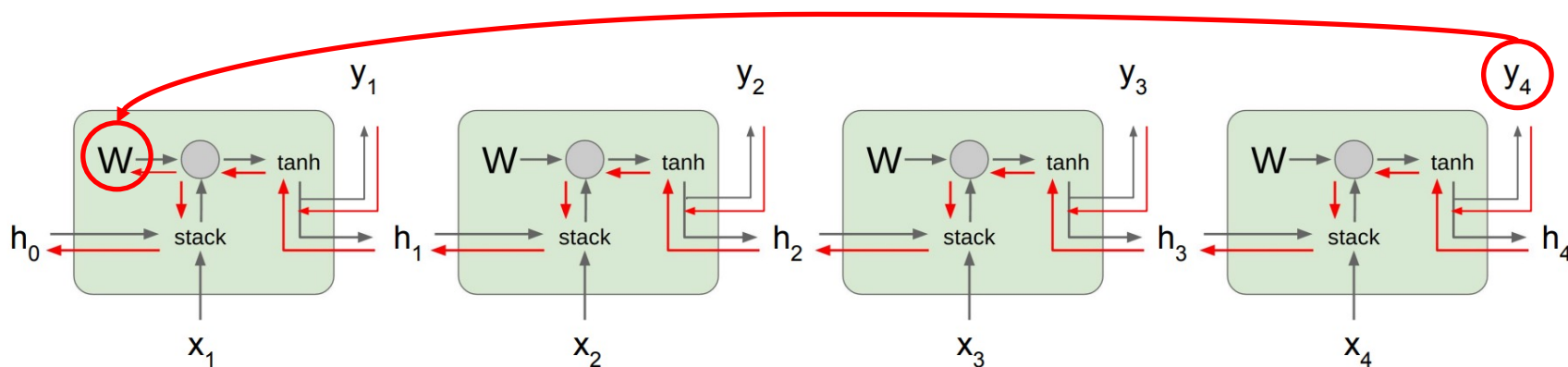
Largest singular value < 1: Vanishing gradients

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \tanh'(W_{hh} h_{t-1} + W_{hx} x_t) \right) \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
    grad *= (threshold / grad_norm)
```


Vanilla RNN Gradient Flow

Gradients over multiple time steps:



Largest singular value > 1: **Exploding gradients**

Largest singular value < 1: **Vanishing gradients** → Change RNN architecture

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \tanh'(W_{hh}h_{t-1} + W_{xh}x_t) \right) \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

Long Short Term Memory (LSTM)

Vanilla RNN

$$\begin{aligned}h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\&= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\&= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)\end{aligned}$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$
$$\begin{aligned}c_t &= f \odot c_{t-1} + i \odot g \\h_t &= o \odot \tanh(c_t)\end{aligned}$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Long Short Term Memory (LSTM)

Vanilla RNN

$$\begin{aligned}h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\&= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\&= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)\end{aligned}$$

LSTM

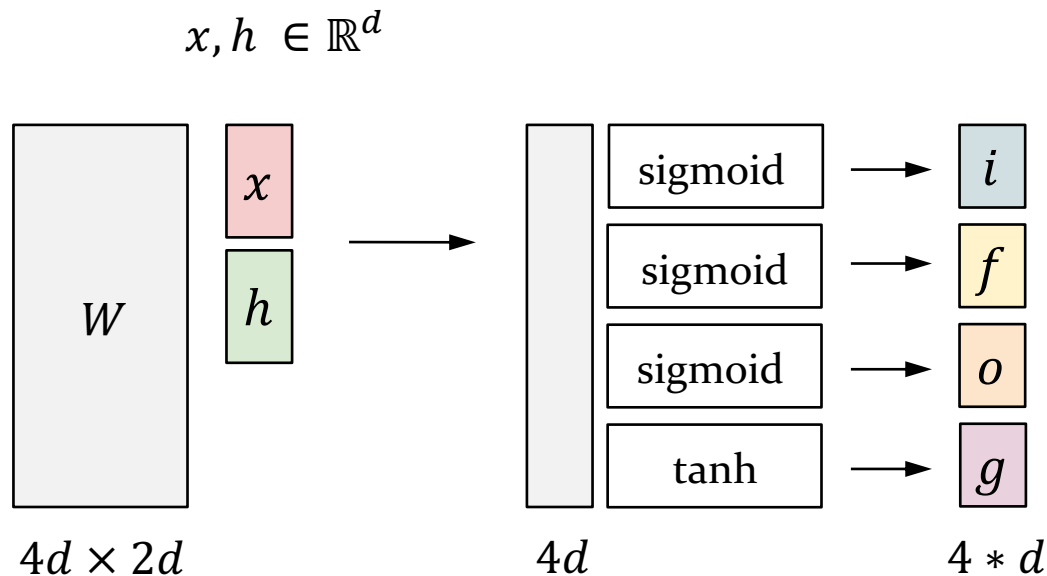
Four gates $\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$

Cell state $c_t = f \odot c_{t-1} + i \odot g$

Hidden state $h_t = o \odot \tanh(c_t)$

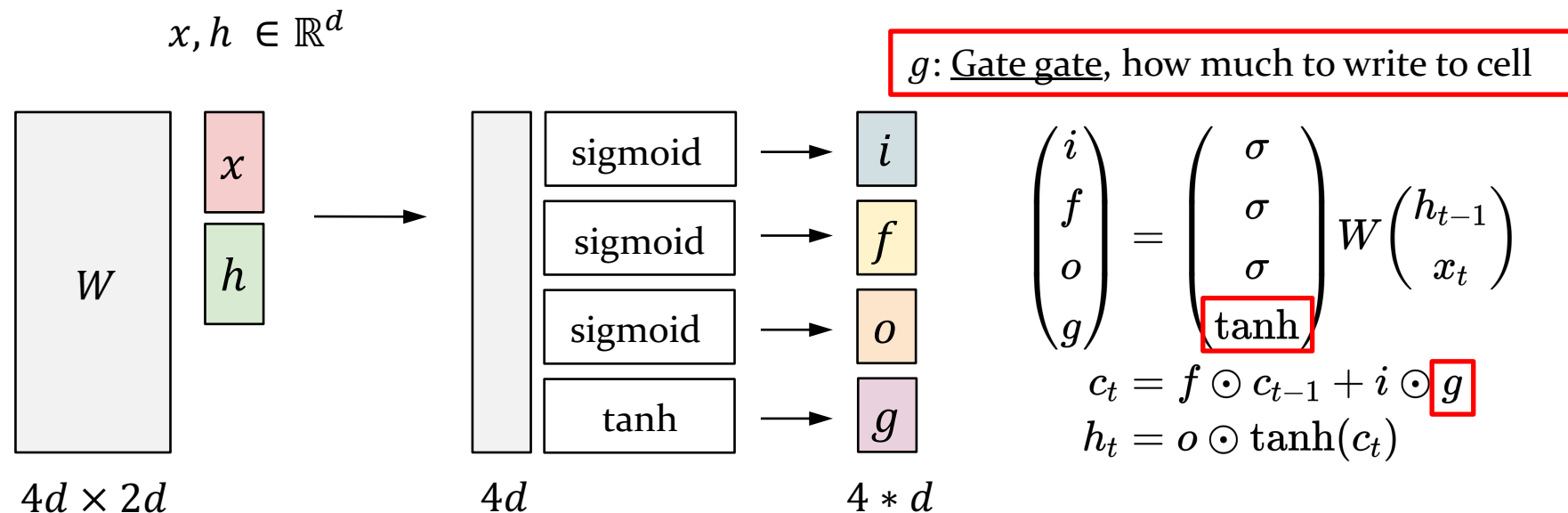
Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Long Short Term Memory (LSTM)



Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

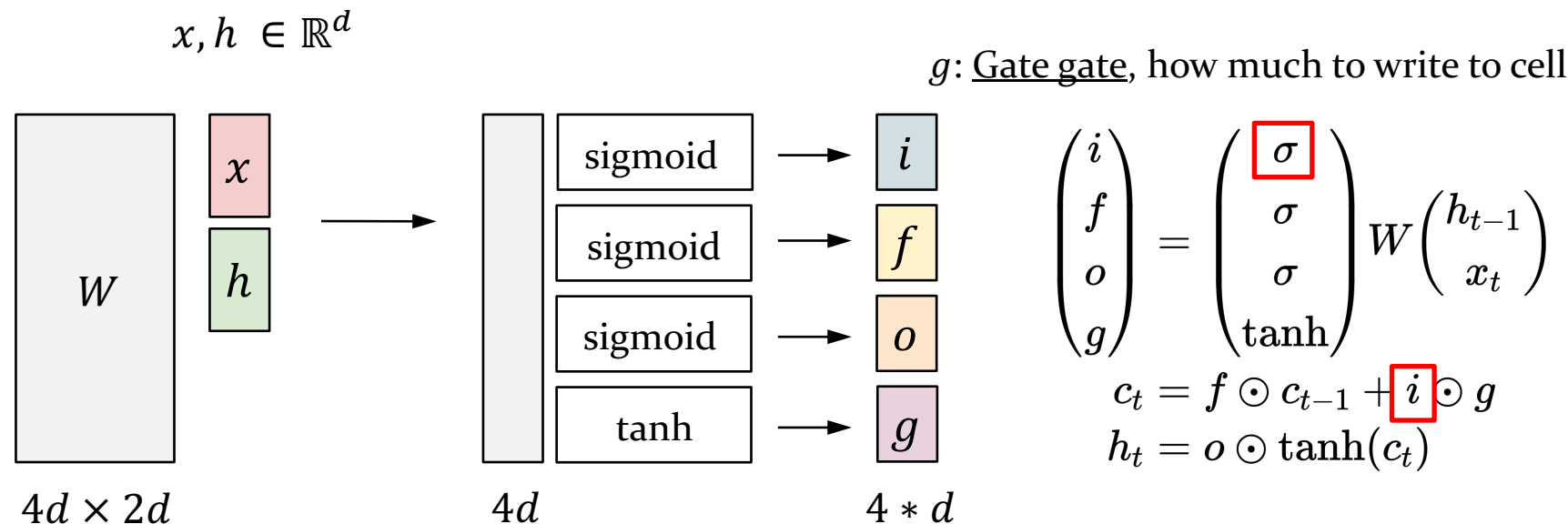
Long Short Term Memory (LSTM)



Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Long Short Term Memory (LSTM)

i : Input gate, whether to write to cell



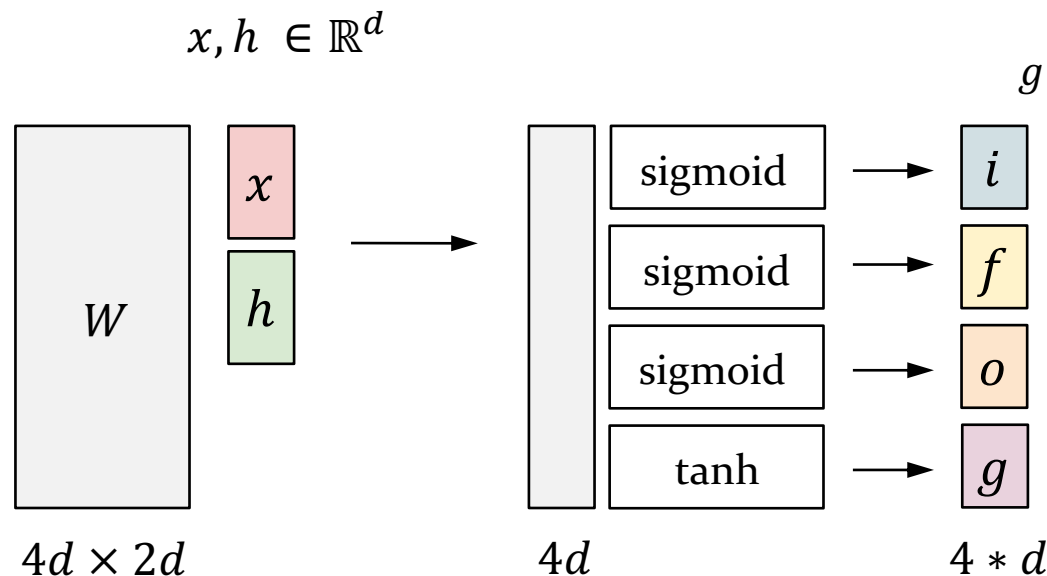
Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Long Short Term Memory (LSTM)

i : Input gate, whether to write to cell

f : Forget gate, whether to erase cell

g : Gate gate, how much to write to cell



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

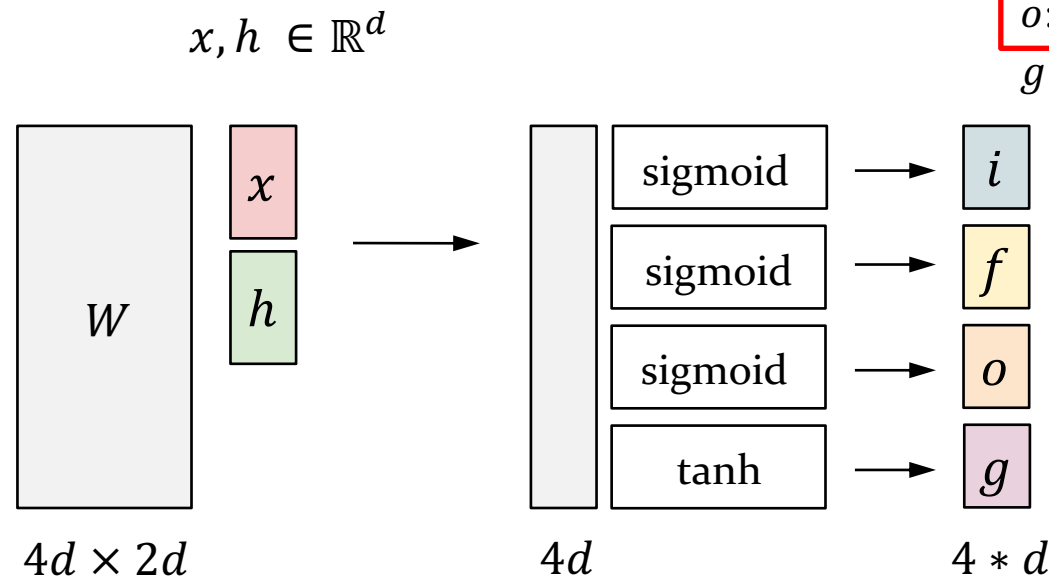
Long Short Term Memory (LSTM)

i : Input gate, whether to write to cell

f : Forget gate, whether to erase cell

o : Output gate, how much to reveal cell

g : Gate gate, how much to write to cell



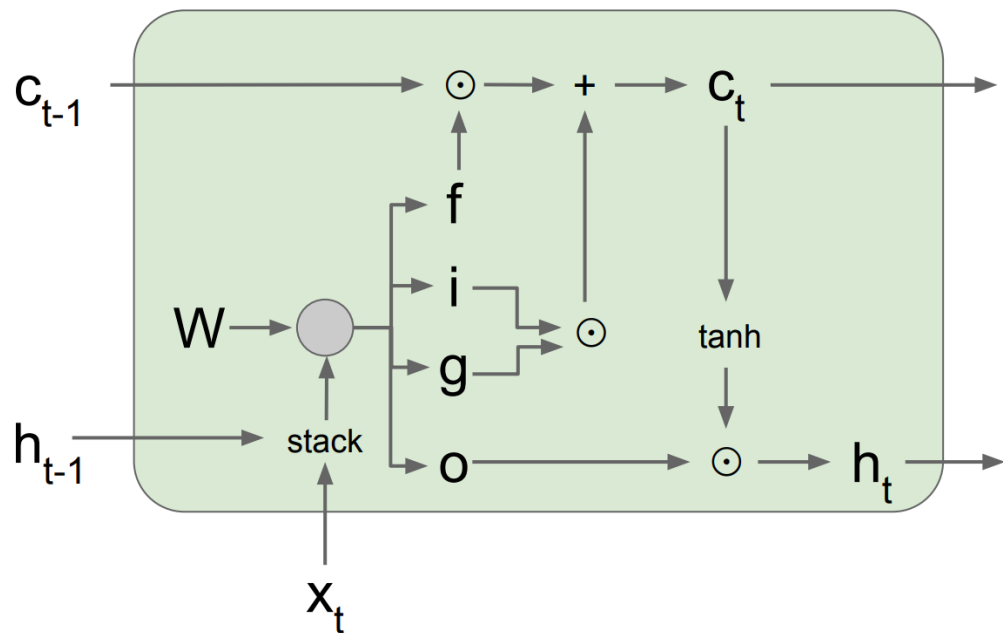
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

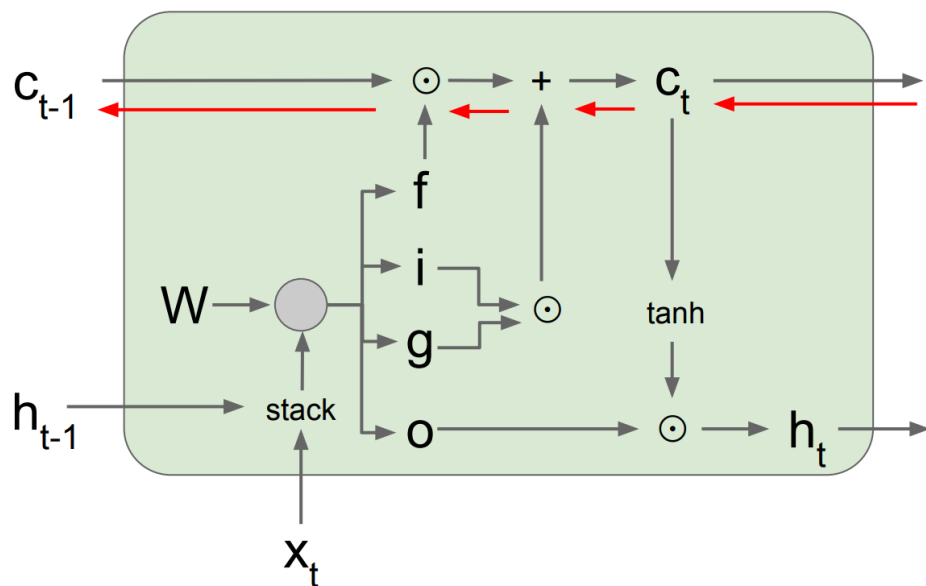
Long Short Term Memory (LSTM)



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$
$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Long Short Term Memory (LSTM) : Gradient Flow



Backpropagation from c_t to c_{t-1} only elementwise multiplication by f , no matrix multiply by W

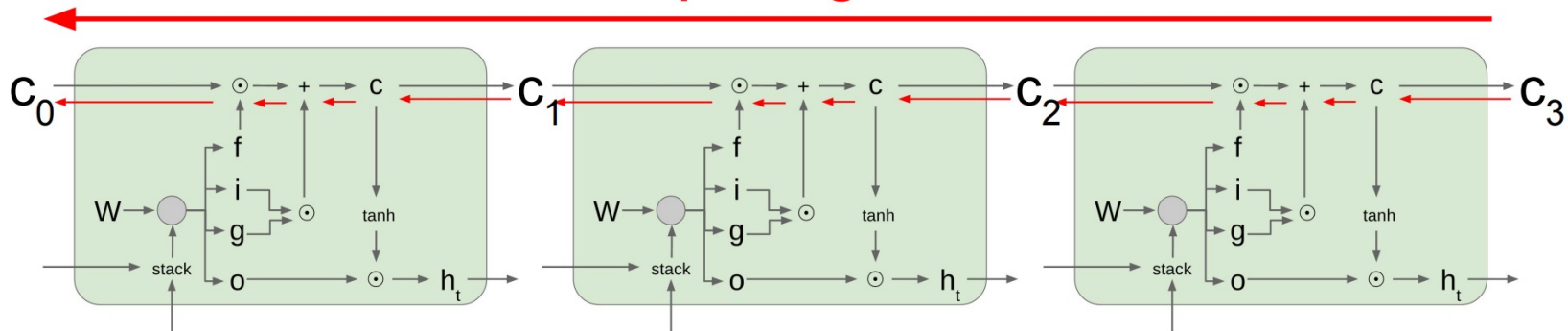
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Long Short Term Memory (LSTM) : Gradient Flow

Uninterrupted gradient flow!



Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Do LSTMs solve the vanishing gradient problem?

- The LSTM architecture makes it easier for the RNN to preserve information over many timesteps
 - e.g. if the $f = 1$ and the $i = 0$, then the information of that cell is preserved indefinitely.
 - By contrast, it's harder for vanilla RNN to learn a recurrent weight matrix Wh that preserves info in hidden state
- LSTM doesn't guarantee that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies

Summary

- RNNs allow a lot of flexibility in architecture design
- Vanilla RNNs are simple but don't work very well
- Backward flow of gradients in RNN can explode or vanish.
- Exploding is controlled with gradient clipping. Vanishing is controlled with additive interactions (LSTM)
- Common to use LSTM or GRU: their additive interactions improve gradient flow
- Better/simpler architectures are a hot topic of current research, as well as new paradigms for reasoning over sequences
- Better understanding (both theoretical and empirical) is needed.